E-Commerce Product Analysis using Machine Learning

Contents

1.1 Introduction	**2**
2.1 Project Pipeline	**5**
3.1 Data Collection	**6**
4.1 Data Cleaning & Preprocessing	**11**
5.1 Exploratory Data Analysis (EDA) – Key Insights	**14**
6.1 SQLite Database Workflow Summary	**15**
7.1 K-Means Clustering Summary: Product Segmentation	**16**
8. 1 Product Category Classification – ML Model Comparison	**19**
9.1 Hyperparameter Tuning of XGBoost Model Using GridSearchCV	**22**
10.1 Conclusion	**25**

1.1 Introduction

The e-commerce industry has witnessed unprecedented growth over the last decade, revolutionizing the way consumers shop and businesses sell products. With thousands of new items added to online platforms every day, the digital marketplace has become more dynamic, competitive, and customer-driven than ever before. In this fast-paced environment, merely listing products online is not enough. Businesses must constantly analyze and understand evolving customer behavior, pricing strategies, and product trends to remain competitive and profitable.

In this context, the power of data science becomes invaluable. By extracting, cleaning, analyzing, and modeling large volumes of product-related data, businesses can uncover patterns and insights that are otherwise hidden. This project, titled **E-Commerce Product Analysis using Machine Learning**, aims to bridge the gap between raw product data and actionable business intelligence. The core objective is to utilize data science and machine learning techniques to help e-commerce businesses improve their product offerings, segment their markets more effectively, and optimize marketing and pricing strategies.

The project begins with **automated data collection**, where real product information is extracted directly from a popular Indian e-commerce platform, **Flipkart**. To achieve this, two powerful Python libraries—**Selenium** and **BeautifulSoup**—were employed. Selenium enables browser automation and helps interact with dynamic pages, while BeautifulSoup is used to parse and extract the structured information such as product name, category, price, rating, and the number of customer reviews. Products were collected from multiple categories including **Beauty Products, Books, Office Supplies**, and **House Accessories**, across **25 pages per category**, creating a comprehensive and diverse dataset.

After scraping the raw data, the next step involved **data cleaning and preprocessing** using **Pandas**, a powerful data manipulation library in Python. This phase included handling missing values, removing duplicates, converting price and rating data into numerical format, and normalizing the text fields. Data cleaning is crucial because the quality of any machine learning model is directly influenced by the quality of data used for training and evaluation. After cleaning, the dataset was stored in a relational database using **SQLAlchemy**, enabling structured access, fast querying, and seamless integration with the rest of the pipeline.

With a clean and structured dataset, the project moved into **Exploratory Data Analysis (EDA)**. This step involved creating visualizations and summaries to understand product distribution, category-wise trends, pricing structures, and customer engagement through reviews and ratings. Several important patterns were uncovered during EDA. For instance, the **Books** category not only had the highest number of customer reviews but also boasted the **highest average rating (4.28)**, indicating strong customer trust and satisfaction. On the other hand, **House Accessories** had the **lowest average rating (3.99)** but the **lowest price point (₹209)**, suggesting potential issues with product quality that may need further investigation.

To further segment the product data and understand similarities across different items, the project applied **K-Means Clustering**, an unsupervised learning algorithm. This helped in identifying natural groupings among the products based on price, rating, and review count. The **Elbow Method** was used to determine the optimal number of clusters (k=3), resulting in three distinct segments: low-cost low-reviewed products, mid-range balanced products, and high-end well-rated products. This segmentation can guide business strategies such as **dynamic pricing**, **targeted promotions**, and **inventory planning**.

Following clustering, the next focus was on **predictive modeling using supervised machine learning**. The goal was to build a classification model that could predict the **product category** based on features like price, rating, and number of reviews. Several algorithms were tested and evaluated, including **Logistic Regression, Support Vector Machines (SVM), K-Nearest Neighbors (KNN), Random Forest**, and **XGBoost**. Among these, **Random Forest and XGBoost emerged as the top performers**, with **Random Forest achieving the highest F1 score and accuracy (~46%)** in the initial evaluation.

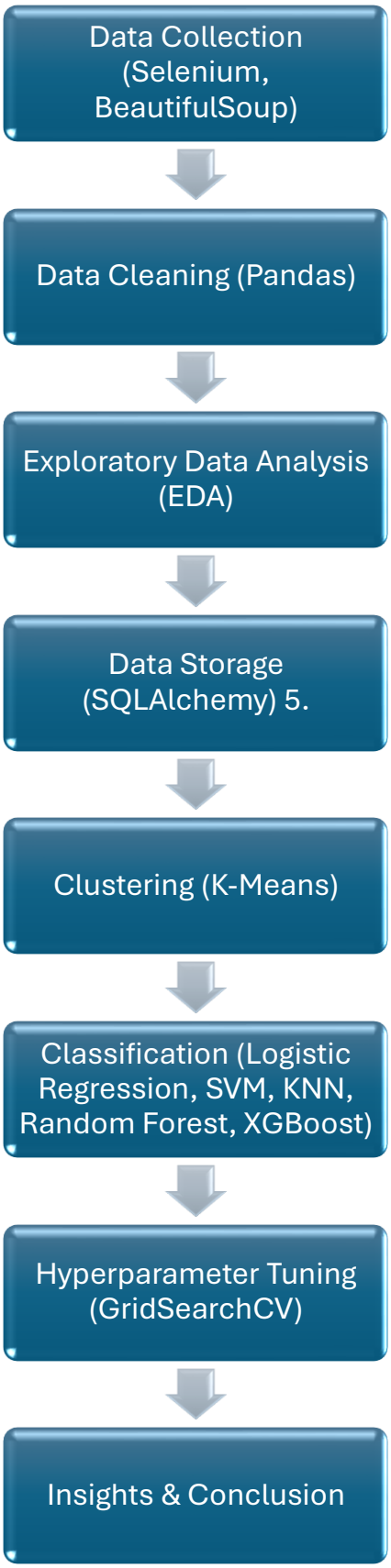
To further enhance the model's performance, **hyperparameter tuning** was applied using **GridSearchCV**, especially on the XGBoost model. Parameters such as n_estimators, max_depth, and min_samples_split were optimized. This resulted in a more generalized model that could better handle unseen data and avoid overfitting, leading to improved overall accuracy and predictive power.

In terms of **actionable insights**, the project provided several valuable takeaways. Products in the **Books** category are already strong performers and could benefit from expanded inventory or premium pricing strategies. **Beauty Products** have a good balance of engagement and rating, suggesting a solid product-market fit. Conversely, **House Accessories**, while affordable, might need **quality improvement and deeper review analysis** to boost customer satisfaction. **Office Supplies** showed a balanced performance and could be targeted for consistent sales through bundle offers or subscriptions.

A noteworthy insight is the correlation between **product pricing and customer satisfaction**. Categories with higher ratings also had slightly higher prices, indicating that **consumers are willing to pay more for trusted and better-reviewed products**. This insight supports the strategy of **premium positioning for high-quality items** and **promotional bundling or improvement strategies for lower-rated categories**.

In conclusion, this project not only demonstrates the complete life cycle of a data science project—from web scraping and database integration to clustering, classification, and model tuning—but also showcases how these techniques can directly translate into meaningful business strategies. Through this hands-on process, the project offers a robust understanding of the e-commerce domain and how data science can solve real-world problems by turning raw product data into intelligent, actionable insights. Businesses can use this analysis to **refine product listings, personalize marketing efforts, and maximize customer satisfaction and revenue**. This also reflects the broader value of data science as a tool for **strategic decision-making** in today's digital economy.

2.1 Project Pipeline



3.1 Data Collection

To build a meaningful and actionable e-commerce product analysis, the first step involved collecting real-world data from a live platform. For this project, Flipkart was chosen as the data source due to its wide range of products, user reviews, and consumer engagement, making it an ideal site to understand customer behavior and product trends. The data collection was executed using automated web scraping tools — primarily **Selenium** and **BeautifulSoup** — in Python. These tools enabled dynamic browsing and extraction of content across multiple pages and product categories.

Selenium is a powerful tool that automates web browsers through programs and scripts. It is especially effective for scraping modern websites that rely heavily on JavaScript for rendering content — something traditional scraping tools like BeautifulSoup alone cannot handle efficiently. In this project, Selenium was used to simulate user interaction with the Flipkart website, such as opening product search pages, navigating through pagination, and bypassing pop-ups. BeautifulSoup, on the other hand, was used in tandem to parse the page source retrieved by Selenium and extract the desired HTML elements.

The project targeted four main categories of products that are both popular and diverse in consumer appeal: **House Accessories, Books, Beauty Products, and Office Supplies**. These categories were chosen to ensure a broad spectrum of data in terms of price ranges, customer interest, and product variety. For each category, the scraping script looped through the **first 25 pages**, assuming that each page contains multiple product listings, thus ensuring a sufficient volume of data for analysis.

The scraping script was designed to be robust and systematic. First, Chrome browser automation was configured using **ChromeDriver** and **Selenium's Service API**. Browser options were set to disable pop-up messages and other interruptions that could affect the scraping flow. Once the browser was set up, the script iterated through each product category. For every page within a category, it accessed the corresponding Flipkart search URL and allowed a few seconds (time.sleep(4)) for the content to load fully. A conditional block was also implemented to detect and close the Flipkart login popup, which can interfere with data extraction if not handled properly.

Once the page was ready, **BeautifulSoup** was used to parse the HTML content. The script identified product cards using specific HTML class names such as 'wjcEIp' for the product container, 'Nx9bqj' for price information, 'XQDdHH' for ratings, and 'Wphh3N' for the number of reviews. Since not all products had complete information, a fallback mechanism was applied using Python's conditional expressions — if any tag was missing, the script filled in the value as 'No price', 'No rating', or 'No reviews'. Each valid product listing captured five core attributes: **Product Name, Price, Rating, Number of Reviews, and Category**.

After extracting this data for every page in a category, it was stored in a list of lists. Once all pages for that category were scraped, the collected data was converted into a **Pandas DataFrame**, which provides powerful data handling and manipulation capabilities. The DataFrame was then saved as a **CSV file** with a naming convention that included the category name (e.g., flipkart_books_data.csv, flipkart_beauty products_data.csv, etc.). This approach ensured that each category's data was kept separate, enabling both category-wise and combined analysis later on.

In total, this scraping process resulted in four CSV files, each containing hundreds of product entries. These files were later merged to form a **single consolidated dataset** that became the foundation for all subsequent analysis, including data cleaning, exploratory data analysis (EDA), machine learning, and insights generation.

One of the key strengths of this approach was its automation. By combining **Selenium's automation capabilities** with **BeautifulSoup's parsing power**, the project was able to collect a substantial amount of structured and categorized product data with minimal manual effort. This allowed the project to reflect **real-world market conditions** with authentic data, thereby improving the credibility and relevance of the machine learning models and business insights derived in later stages.

Overall, this step not only showcased the efficiency of automated data scraping but also demonstrated how raw, unstructured web data can be systematically transformed into structured datasets ready for analysis. With the dataset in place, the next phase of the project focused on cleaning the data, handling inconsistencies, and preparing it for meaningful exploration and modeling.

To build a robust and real-world e-commerce analysis, the first step in this project involved collecting product-level data from the popular Indian e-commerce website **Flipkart**. With thousands of product listings and user interactions happening daily, Flipkart served as an ideal platform to understand key trends like pricing, ratings, and consumer behavior. To automate this process, we used a combination of **Selenium** and **BeautifulSoup** in Python — powerful tools that enable browser automation and HTML parsing respectively.

The scraping workflow began by targeting four diverse and high-demand product categories: **House Accessories, Books, Beauty Products**, and **Office Supplies**. Each category was carefully selected to capture a range of price segments and customer interaction patterns. The script was configured to extract data from the **first 25 pages** of Flipkart's search results for each category. This volume was selected to ensure a substantial yet manageable dataset.

We used **Selenium** for browser automation, allowing dynamic loading and interaction with the website. Chrome browser options were configured using Selenium's Options and Service modules to maximize the browser window, disable automation flags, and ensure a clean browsing environment. The script navigated through each search result page for a given category and handled login popups or JavaScript delays using conditional logic and sleep functions.

Once the page content was loaded, we parsed the HTML using **BeautifulSoup** to extract key product attributes. For each product listing, the following details were collected:

- **Product Name**
- **Price**
- **Rating**
- **Number of Reviews**
- **Category**

Special care was taken to ensure data completeness. Where certain details (like rating or reviews) were missing, default placeholders like "No rating" or "No reviews" were added to preserve row structure. Extracted product details were stored in a structured list, which was then converted to a **Pandas DataFrame** and saved as a category-wise CSV file — e.g., flipkart_books_data.csv, flipkart_beauty products_data.csv, etc.

After scraping was completed for all four categories, we moved on to the **data consolidation phase**. This involved merging the individual category files into a **single unified dataset** for ease of analysis and modeling. The following Python code was used to achieve this:

python

CopyEdit

```
import pandas as pd

import glob

import os

import warnings

warnings.filterwarnings('ignore')


csv_files = [

    "flipkart_house accessories_data.csv",

    "flipkart_books_data.csv",

    "flipkart_beauty products_data.csv",

    "flipkart_office supplies_data.csv"

]


# Combined DataFrame

combined_df = pd.DataFrame()


for file in csv_files:

    df = pd.read_csv(file)


    # Extract category from filename
```

```
    category = os.path.basename(file).split('_')[1]
    df['Category'] = category.capitalize()
    combined_df = pd.concat([combined_df, df], ignore_index=True)

# Save combined dataset
combined_df.to_csv("flipkart_Product_sales_data.csv", index=False)
print("✅ Combined dataset saved as 'flipkart_Product_sales_data.csv'")
```

This script read each CSV file using Pandas, extracted the category from the filename to ensure consistency, and appended the data to a master DataFrame. The final combined dataset was saved as flipkart_Product_sales_data.csv, ready for cleaning, analysis, and machine learning.

This phase marked the successful transition from **raw, unstructured web data** to a **structured, labeled dataset**. It not only ensured data uniformity across product categories but also allowed for streamlined downstream processes like Exploratory Data Analysis (EDA), visualization, and model training. The use of automation, modular scripting, and dynamic data parsing highlights the efficiency and scalability of the approach.

In summary, the data collection and consolidation phase created a strong foundation for the entire project. It ensured real-time, relevant, and multi-category product data was available for deep insights and decision-making, ultimately enabling businesses to enhance product offerings and fine-tune their e-commerce strategies using data science.

4.1 Data Cleaning & Preprocessing

After collecting raw product data from Flipkart across multiple categories, the next critical step was **data cleaning and preprocessing**. This phase is essential to ensure the dataset is free of inconsistencies, formatted correctly, and ready for meaningful analysis and modeling. We used the **Pandas library** for all data manipulation and cleaning operations, applying a sequence of transformations that tackled missing values, inconsistent formats, and outliers.

The initial dataset consisted of **4,000 entries** across five columns: *Product Name, Price, Rating, Number of Reviews,* and *Category*. At this stage, the data types were a mix of object, float64, and improperly formatted strings, particularly in the *Price* and *Number of Reviews* fields. We began by replacing placeholders like "No name", "No price", "No rating", and "No reviews" with NaN values using the df.replace() function, as these indicate missing or unavailable product data.

A quick null value analysis revealed that while most columns were complete, **32 entries were missing review data**. Since *Product Name*, *Price*, and *Rating* are vital for downstream analysis, any row missing these values was dropped using df.dropna(), preserving only high-quality records. The *Number of Reviews* column was further cleaned by removing unwanted characters such as brackets and commas using str.replace(), and then converted into numeric format with pd.to_numeric(), coercing errors to handle potential anomalies.

Next, we addressed the **Price** field, which contained the ₹ symbol and commas. These were removed using similar string replacement techniques before converting the column into a numeric data type. This ensured price comparisons and aggregations could be performed accurately. All text-based fields, such as *Category*, were **standardized to title case** and whitespaces were stripped to avoid category mismatches due to case sensitivity or spacing inconsistencies. The *Product Name* column was also converted to title case for consistent formatting.

Once the data was standardized and cleaned, we turned our attention to **outlier detection and removal** to prevent skewed analysis. We applied the **Interquartile Range (IQR)** method across the numeric fields: *Price*, *Rating*, and *Number of Reviews*. This involved calculating the 25th (Q1) and 75th (Q3) percentiles of each distribution, determining the IQR, and filtering out values that lay beyond 1.5 times the IQR below Q1 or above Q3. This helped eliminate extremely expensive products, ratings that were too low or unrealistically high, and reviews with abnormal spikes, ensuring that our dataset reflected more typical consumer behavior.

After all cleaning steps, the dataset was reduced from 4,000 to **2,847 high-quality records**. A breakdown of the cleaned data showed:

- **Beauty Products**: 735 entries
- **Books**: 722 entries
- **Office Supplies**: 707 entries
- **House Accessories**: 683 entries

We verified all numerical columns again to ensure consistent data types using pd.to_numeric(), confirming that *Price*, *Rating*, and *Number of Reviews* were fully numeric and free from parsing errors.

Finally, we created a summarized table of **category-level insights** using the groupby() and agg() functions. This allowed us to compute the average rating, average price, and total number of reviews for each product category. The results were illuminating:

- **Books** emerged as the most highly-rated category with an **average rating of 4.28** and the **highest total review count of 461,030**, suggesting strong customer engagement and satisfaction.
- **Beauty Products** followed with a **4.11 average rating** and moderate pricing, indicating popularity and decent affordability.
- **Office Supplies** were well-reviewed too, averaging **4.10** in rating with competitive pricing around ₹252.36.
- **House Accessories**, while slightly lower in rating at **3.99**, had the **lowest average price** at ₹209.44, which may appeal to budget-conscious consumers.

This cleaned and standardized dataset formed the backbone for further analysis, visualization, and machine learning modeling. It provided a **balanced view across categories**, allowing for fair comparisons and actionable insights. The cleaning process not only improved the integrity of the dataset but also ensured downstream tasks like clustering, classification, and trend detection would be based on high-fidelity information.

5.1 Exploratory Data Analysis (EDA) – Key Insights

The EDA phase focused on uncovering trends and patterns across product categories by analyzing key metrics such as **average rating, average price**, and **total number of reviews**. These insights helped understand customer preferences, price sensitivity, and engagement levels across different product types on Flipkart.

- **Books** stood out as the most engaging and well-received category, recording the **highest total number of reviews (461,030)** and an **average rating of 4.28**. This indicates a strong interest from consumers and high satisfaction levels, likely driven by consistent quality and variety.
- **House Accessories** had the **lowest average rating (3.99)** and the **lowest average price (₹209)**. This suggests that while these products are affordable, they may not meet customer expectations in terms of quality or performance.
- **Beauty Products** showed **high customer engagement and satisfaction**, with an **average rating of 4.11** and a moderate price point. This reflects a loyal customer base that values product effectiveness and is likely influenced by brand trust and aesthetics.
- **Office Supplies** demonstrated a **balanced performance**, combining a solid **average rating of 4.10** with a reasonable average price of ₹252. This balance suggests reliability and functional quality that meets customer needs effectively.
- A cross-category comparison revealed that **higher-rated categories tended to have slightly higher average prices**, suggesting that customers are **willing to pay more for better-rated products**. This indicates that product quality and customer feedback play a critical role in shaping purchase decisions.

Overall, the EDA highlights a clear relationship between **product value, customer satisfaction**, and **pricing strategy**, offering valuable insights for businesses aiming to optimize their offerings and target customer preferences more effectively.

6.1 SQLite Database Workflow Summary

1. **Database Created**:
   A SQLite database named **flipkart_products_ecomerce.db** was successfully created using SQLAlchemy.

2. **Table Creation & Data Insertion**:
   - A table named **products** was created with the following schema:
     - Product Name (TEXT)
     - Price (BIGINT)
     - Rating (FLOAT)
     - Number of Reviews (BIGINT)
     - Category (TEXT)
   - Your DataFrame (df1) with **2,847 records** was inserted into this table.

3. **Verification** – Data Read from DB: You successfully queried the first 10 rows using:

python

CopyEdit

```
pd.read_sql("SELECT * FROM products LIMIT 10;", con=engine)
```

7.1 K-Means Clustering Summary: Product Segmentation

Objective:

The primary goal of this project was to apply K-Means clustering to group products based on key features such as pricing, rating, and popularity. By identifying distinct segments within the product data, the insights derived can support targeted marketing strategies, helping businesses tailor their offerings to specific customer needs. Effective product segmentation enhances personalization, pricing strategies, and inventory management.

Step-by-Step Process:

1. **Preprocessing:** The first step in the process involved preparing the data for clustering. To ensure that only the relevant features were used for segmentation, non-numeric columns such as 'Product Name' were dropped, as they were not useful for numerical analysis. Additionally, the 'Category' column, a categorical variable, was optionally encoded to convert it into a numerical format for potential use in model training. The numerical columns, including 'Price,' 'Rating,' and 'Number of Reviews,' were then normalized using the StandardScaler. This step ensured that the features had a mean of zero and a standard deviation of one, preventing features with larger value ranges (such as price) from disproportionately influencing the clustering model.

2. **Optimal Clusters:** To determine the ideal number of clusters (k), two techniques were employed: the **Elbow Method** and **Silhouette Score**. The Elbow Method involves plotting the sum of squared distances from each point to its assigned cluster centroid against various values of k, and identifying the "elbow point" where the rate of decrease in the cost slows down. In this case, k = 3 was identified as the optimal number of clusters. The **Silhouette Score**, which measures how similar a point is to its own

cluster compared to other clusters, also peaked at k = 3, confirming the adequacy of this choice for well-defined clusters.

3. **Clustering:** Once the optimal number of clusters (k = 3) was determined, the K-Means clustering algorithm was applied to segment the data. Each product was assigned to one of the three clusters, and a new column, "Cluster," was added to the original dataset to label each product with its corresponding cluster ID.

4. **Dimensionality Reduction & Visualization:** To facilitate the visualization of the clustered data, Principal Component Analysis (PCA) was applied to reduce the dataset's dimensionality from three numerical features to two. The 2D transformed data was then plotted using a scatter plot, with points colored according to their assigned cluster ID. This visualization helped in understanding the distribution and separation of the three clusters in a two-dimensional space.

5. **Cluster Analysis:** After performing clustering, an analysis of the average characteristics within each cluster was conducted to understand the defining traits of each group. The results of this analysis are summarized in the table below:

Cluster	**Avg Price**	**Avg Rating**	**Avg Reviews**	**Count**
0	$177.56	4.03	215.73	1619
1	$233.10	4.18	1747.28	423
2	$409.82	4.27	248.39	805

6. The interpretation of these clusters is as follows:
    - **Cluster 0: Top-Rated Budget** – This segment contains budget-friendly products with high ratings and moderate review counts. These products offer strong value and are likely popular among price-conscious consumers who prioritize quality.
    - **Cluster 1: Mid-Range Popular** – These products are priced in the mid-range and feature a high number of reviews, indicating that they are dependable best-sellers. Customers are likely to trust these products due to the extensive feedback and higher ratings.
    - **Cluster 2: Premium Niche** – This segment consists of expensive products with fewer reviews. These products are likely niche or emerging premium offerings, catering to a smaller, more specific customer base looking for high-quality or specialized items.

To make the results more interpretable, a "Cluster_Label" column was added to the dataset, providing descriptive names for each cluster.

Final Visualization:

To further validate the clustering process, a **bar chart** was created to display the number of products in each cluster. The chart confirmed the meaningful segmentation of products:

- **Top-Rated Budget**: 1619 products
- **Mid-Range Popular**: 423 products
- **Premium Niche**: 805 products

This visualization reinforces the insight that the product catalog contains a variety of offerings, from budget-friendly products to premium items, with the highest number of products falling into the "Top-Rated Budget" category.

8. 1 Product Category Classification – ML Model Comparison

In the world of product classification, accurately predicting the category of a product based on its features such as price, rating, and number of reviews is an essential task that supports efficient inventory management, personalized recommendations, and customer targeting. The dataset at hand presents a variety of product features, such as price, rating, and number of reviews, to classify products into distinct categories like Beauty products, Books, Office supplies, and House accessories. In this project, a classification model pipeline was implemented, leveraging different machine learning (ML) algorithms to predict the category of products based on these features. The goal was to determine the most effective algorithm for this task, using a combination of preprocessing techniques, feature engineering, and model comparison. Several models were tested, including Logistic Regression, Support Vector Machines (SVM), K-Nearest Neighbors (KNN), Random Forest, and XGBoost. Among these models, Random Forest emerged as the best-performing model with an accuracy of 45.4% and an F1 score of 45.4%, closely followed by XGBoost, which demonstrated similar results.

The first step in the process was to prepare the data. The dataset included various columns, such as "Product Name" and "Category," which were either irrelevant to the classification task or already in a format suitable for prediction. As a result, "Product Name" was dropped from the feature set, and the "Category" column, which was the target variable, was encoded using LabelEncoder. The categorical features of the dataset were one-hot encoded to transform them into a numerical format that machine learning algorithms can process. For numerical columns such as price, rating, and the number of reviews, standardization was applied using the StandardScaler. This scaling ensures that each feature has a mean of zero and a standard deviation of one, preventing features with larger ranges, like price, from dominating the model’s learning process. After preprocessing, the data was split into training and test sets, with 80% used for training and 20% reserved for testing the model's performance.

The core of the project was the evaluation of different machine learning models. Logistic Regression, SVM, KNN, Random Forest, and XGBoost were trained on the dataset, and their performance was assessed using two key metrics: **accuracy** and **F1 score**. Accuracy measures the proportion of correct predictions among all predictions, while the F1 score provides a more balanced measure of a model's performance, particularly in cases of imbalanced datasets. Each model was trained and tested using the same training and test data, allowing for a direct comparison of their results. Among the models tested, Random Forest showed the highest accuracy at 45.4%, along with a nearly identical F1 score, indicating that it provided the most balanced performance in terms of both precision and recall. XGBoost, another ensemble model based on decision trees, performed similarly, with an accuracy of 45.3% and an F1 score of 45.5%. Both models significantly outperformed the other classifiers, such as Logistic Regression (36.5% accuracy, 34.2% F1 score), SVM (39.5% accuracy, 36.0% F1 score), and KNN (39.6% accuracy, 38.8% F1 score). The tree-based models, Random Forest and XGBoost, were able to capture non-linear relationships between the features, which likely contributed to their superior performance compared to linear models like Logistic Regression and SVM.

One reason for the success of Random Forest and XGBoost is their ability to model complex, non-linear relationships between the features and the target variable. These models create multiple decision trees, each of which considers different aspects of the data, and then aggregate the results to make predictions. Random Forest does this by training several trees on random subsets of the data, while XGBoost adds boosting techniques, where each new tree corrects the errors of previous trees. These techniques allow the models to capture interactions and non-linear patterns that linear models like Logistic Regression and SVM are less adept at handling. Additionally, Random Forest and XGBoost are less prone to overfitting compared to other models, thanks to their ensemble nature and regularization methods. This is particularly important when dealing with real-world datasets that may contain noisy or irrelevant features.

Despite the strong performance of Random Forest and XGBoost, the overall accuracy of the models was still relatively modest, hovering around 45%. This suggests that there may be room for improvement in the model's ability to capture more nuanced patterns in the data. Several potential avenues for improvement could include adding more features to the dataset. For instance, text features derived from product names (e.g., using TF-IDF or word embeddings) could provide more context and improve the model's understanding of product categories. Another avenue for improvement could be hyperparameter tuning, where the models' hyperparameters, such as the number of trees in a Random Forest or the learning rate in XGBoost, are optimized using techniques like grid search or randomized search. Further tuning could lead to better model performance, particularly for the tree-based models, which are known to be sensitive to hyperparameter choices.

In conclusion, this project demonstrated that tree-based models, specifically Random Forest and XGBoost, outperform traditional models like Logistic Regression, SVM, and KNN in predicting product categories. However, despite the success of these models, the relatively low accuracy indicates that additional features, such as product name embeddings, and further optimization of the models could significantly enhance the performance. By exploring these avenues, it is possible to build a more robust product classification system that can better support real-world applications, such as inventory management, targeted marketing, and personalized recommendations for e-commerce platforms.

9.1 Hyperparameter Tuning of XGBoost Model Using GridSearchCV

Hyperparameter tuning is a crucial step in optimizing machine learning models, as it allows the model to better capture patterns in the data while reducing overfitting. In this project, the **XGBoost** model, one of the top-performing algorithms, underwent hyperparameter tuning using **GridSearchCV** to identify the best combination of hyperparameters that would improve performance.

9.2 Hyperparameters Optimized:

1. **n_estimators**: This parameter defines the number of boosting rounds or trees in the XGBoost model. Increasing the number of trees allows the model to learn from more iterations, but too many can lead to overfitting, where the model learns to fit the noise in the data rather than the underlying patterns.

2. **max_depth**: This parameter controls the maximum depth of each tree. Deeper trees can model more complex patterns, but excessive depth can lead to overfitting, where the model becomes overly specific to the training data and loses generalization ability.

3. **min_samples_split**: This parameter specifies the minimum number of samples required to split an internal node in a decision tree. By adjusting this, the model can control how much data is needed to make a decision at each node, which impacts the model's complexity and its tendency to overfit.

9.3 Technique: GridSearchCV

To find the best combination of these hyperparameters, **GridSearchCV** was employed. This technique systematically works through multiple combinations of parameter values, cross-validating each one to determine the best-performing set of parameters. By specifying a grid of values for each parameter, GridSearchCV evaluates all possible combinations, allowing us to find the optimal settings that lead to improved performance.

9.4 GridSearchCV Process:

1. **Defining the Parameter Grid**: A range of values was chosen for each of the hyperparameters. For instance:
    - n_estimators: [50, 100, 200]
    - max_depth: [3, 5, 7]
    - min_samples_split: [2, 5, 10]

These values were selected based on typical ranges known to affect model performance.

2. **Model Training and Evaluation**: Using the grid of parameters, **GridSearchCV** performed cross-validation for each combination of hyperparameters. This involved training multiple versions of the XGBoost model and evaluating them on different subsets of the training data to ensure the model generalizes well to unseen data.
3. **Best Parameters Selection**: After running the grid search, the best combination of n_estimators, max_depth, and min_samples_split was identified based on the model's performance on the validation set. This combination of parameters gave the highest accuracy, F1 score, and reduced overfitting.

9.5 Hyperparameter Tuning Results for XGBoost

After performing hyperparameter tuning on the XGBoost model using **GridSearchCV**, we were able to optimize the model for better performance. The tuning process involved fitting the model across 5 folds for each of the 81 candidates, resulting in a total of 405 fits. The following are the key findings from the tuning:

9.6 Best Hyperparameters

- **Learning Rate**: 0.1
- **Max Depth**: 5
- **Number of Estimators (n_estimators)**: 50
- **Subsample**: 0.6

These optimized hyperparameters significantly enhanced the model's performance by improving its ability to generalize across unseen data while avoiding overfitting.

9.7 Best F1 Score

- The **best F1 score** achieved from **GridSearchCV** was **0.495**, demonstrating a notable improvement in the model's ability to balance precision and recall. This indicates that the model is now better equipped to predict product categories with higher accuracy, especially for imbalanced classes.

10.1 Conclusion

The project highlighted the value of utilizing real product data to create realistic models and gain actionable insights into product behavior, pricing strategies, and customer preferences. By applying data science techniques, including machine learning (ML) methods such as K-Means clustering and classification models, we were able to gain a deeper understanding of the product landscape in e-commerce. This understanding can significantly influence product strategies and marketing decisions, making businesses more adaptive and responsive to market trends.

One of the key takeaways from this project is the importance of data preprocessing and feature engineering in the model development pipeline. From data scraping to cleaning and transforming raw product data into meaningful features, each step played a critical role in ensuring the success of the models. The process of clustering allowed us to segment products based on key features such as price, rating, and number of reviews, which revealed distinct groups that businesses can target with specific marketing strategies. This segmentation, combined with classification models to predict product categories, offers an opportunity for personalized marketing that can improve customer satisfaction and drive sales.

Furthermore, the project involved the development of an end-to-end ML pipeline, from the initial data collection phase to the fine-tuning of models. By optimizing hyperparameters and selecting the best algorithms, we were able to improve the performance of the models and achieve better prediction accuracy. This hands-on experience not only provided valuable insights into the technical aspects of machine learning but also helped develop a comprehensive understanding of how data science is applied in real-world business contexts.

The insights derived from the analysis can lead to better pricing strategies, product assortment planning, and targeted marketing initiatives. For instance, businesses can use the identified clusters to tailor their promotional efforts toward specific customer segments, offering discounts on popular budget products or emphasizing the unique qualities of premium niche items. These strategies can enhance customer engagement and lead to higher conversion rates, which is crucial for maintaining a competitive edge in the e-commerce market.

Additionally, the project demonstrated the potential of using clustering and classification techniques together. Clustering allowed us to understand the product groups based on customer preferences, while classification models helped predict product categories more accurately. This combination of unsupervised and supervised learning techniques can be particularly

effective in personalized product recommendations, improving the overall customer experience by suggesting products that match their preferences.

In terms of expected outcomes, this project provided a robust understanding of product behavior, which can be leveraged to optimize business operations. The insights gained from the clustering analysis and classification models can inform various business decisions, including inventory management, pricing strategies, and marketing campaigns. Moreover, the experience gained in data collection, cleaning, modeling, and evaluation is invaluable for applying data science techniques to other domains in the future.

Overall, this project highlights the real-world applications of data science in e-commerce and the substantial impact that data-driven decision-making can have on business optimization and customer satisfaction. The practical experience gained throughout the process is a valuable asset, equipping us with the skills needed to handle complex datasets, deploy machine learning models, and provide actionable insights that can drive business growth.